

Problem Statement: Build a Smart Mutual Fund Advisor

Name: Kushagra Gupta

Roll No: 27PGAI0115

On top of the Mutual Funds Report, [sourced](#) from Kaggle

Objective

Create a Gradio-based AI assistant that recommends a mutual fund investment portfolio based on the user's risk and return preferences. The app uses Retrieval-Augmented Generation (RAG) on a structured Mutual Fund product dataset, generating explainable, structured results.

User App (Essential Features)

Inputs

- **Investment Needs:** Free-form text input
(e.g., "high return portfolio, only from Aditya Birla fund house")

Output

- **Suggestion:** A natural language explanation of what's recommended and why
 - **Recommended Mutual Funds:** Structured as a list containing:
 - Scheme Name
 - Fund House
-

Developer-Facing Advanced Settings

Shown under a collapsible **LLM Settings (Advanced)** section in the UI.

Model & Generation Settings

- **Model Selector:** select 2 more open-source / closed-source models
 - GPT OSS 120B (gpt-oss-120b)
- **Temperature Slider:** Adjustable between 0 and 2

These settings allow developers to tune how deterministic or creative the model's responses are.

Backend Setup

Vector Store and RAG

- **Embedding Model:** Nomic Embed Text via Ollama, or other model of your choice
- **Vector Database:** ChromaDB, or other vector DB of your choice
- **RAG Workflow:**
 1. Load product catalog using `CSVLoader`
 2. Parse and clean fields into `Document` objects with metadata
 3. Generate embeddings and index documents
 4. Perform similarity search based on user preferences
 5. Use an LLM to generate structured output parsed via `PydanticOutputParser`

Part 1: Implement the Solution

Setup

```
In [1]: import gradio as gr
import pandas as pd
import os
from dotenv import load_dotenv
from IPython.display import Image, display
from langchain_tavily import TavilySearch
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings
from langchain_community.document_loaders import CSVLoader
from langchain.chat_models import init_chat_model
from langchain_openai import OpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.output_parsers import PydanticOutputParser
from langchain.schema.output_parser import OutputParserException
from langchain_core.documents import Document
from pydantic import BaseModel, Field
from uuid import uuid4
from langgraph.graph import START, StateGraph, END
from typing_extensions import List, TypedDict
from langgraph.checkpoint.memory import MemorySaver
```

```
In [2]: ## TODO: Load environment variables and API Keys here
load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
```

Defining Components

Chat Model

```
In [3]: ## TODO: Model name map for UI Mapping
model_name_map = {
```

```

    "GPT-OSS-120B": "openai/gpt-oss-120b",
    ## TODO: Add 2 more models here
    "GPT-OSS-20B": "openai/gpt-oss-20b",
    "Qwen3.8-27B": "qwen/qwen3.8-27b"
}

model_choices = [
    "GPT-OSS-120B (Groq)",
    ## TODO: Add 2 more models here
    "GPT-OSS-20B (Groq)",
    "Qwen3.8-27B (Groq)"
]

```

```

In [4]: ## TODO: Model Selector
def get_llm(model_choice, temperature):
    if "Groq" in model_choice:
        return init_chat_model(model_name_map[model_choice.split(" ")[0]],
                                model_provider="groq", temperature = temperature)
    else:
        raise ValueError("Invalid model")

```

Embedding Model

```

In [5]: ## TODO: Create embeddings
## Use: OllamaEmbeddings with nomic-embed-text via Ollama
## (Llama3.2:1b is a chat model, not tagged for the /api/embed endpoint on
## recent Ollama versions)
embeddings = OllamaEmbeddings(model="nomic-embed-text",
                                base_url="http://localhost:11434")

```

Vector Store

```

In [6]: ## TODO: Initialize ChromaDB with a persist directory set
vector_store = Chroma(
    collection_name="mutual_funds",
    embedding_function=embeddings,
    persist_directory="./chroma_db"
)

```

Load and Preprocess Data

```

In [7]: # Load and preprocess data
loader = CSVLoader(file_path="mutual_funds_data.csv",
                    source_column="scheme_code")
documents_raw = loader.load()

```

```

In [8]: # Convert page_content string to dict and build metadata
documents = []
for doc in documents_raw:
    try:
        row_data = dict(
            line.split(":", 1) for line in doc.page_content.split("\n") if ":"
in line
        )
        row_data = {k.strip(): v.strip() for k, v in row_data.items()}

```

```

        ## TODO: Keep Scheme Name, Fund House, Scheme Type, Scheme Category,
        Net Asset Value and Date in content
        page_text = f"Scheme Name: {row_data.get('scheme_name', '')} | Fund
        House: {row_data.get('fund_house', '')} | Scheme Type:
        {row_data.get('scheme_type', '')} | Scheme Category:
        {row_data.get('scheme_category', '')} | Net Asset Value:
        {row_data.get('net_asset_value', '')} | Date: {row_data.get('date', '')}"

        ## TODO: Keep Scheme Type, Scheme Category in metadata
        metadata = {
            "scheme_type": row_data.get("scheme_type", ""),
            "scheme_category": row_data.get("scheme_category", "")
        }

        documents.append(Document(page_content=page_text, metadata=metadata))

    except Exception as e:
        print("Skipping row due to error:", e)

```

In [9]: len(documents)

Out[9]: 35559

In [10]: print(documents[0].page_content)

Scheme Name: Grindlays Super Saver Income Fund-GSSIF-Half Yearly Dividend | Fund House: Standard Chartered Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Income | Net Asset Value: 10.7205 | Date: 29-05-2008

In [11]: len(documents)

Out[11]: 35559

Store in Vector DB

```

In [12]: ## Store in Chroma - 1000 rows only for quicker processing - and make sure to
        add UUIDs
        ## TODO: Add UUIDs
        uuids = [str(uuid4()) for _ in range(len(documents[:1000]))]

        ## TODO: Add documents
        batch_size = 100
        for i in range(0, 1000, batch_size):
            vector_store.add_documents(documents=documents[i:i + batch_size],
            ids=uuids[i:i + batch_size])

```

Initializing for RAG

Initialize Retriever

```

In [13]: ## TODO: Initializer Retriever below
        retriever = vector_store.as_retriever(search_kwargs={"k": 8})

```

Define Pydantic Schema

```
In [14]: ## TODO: Define Pydantic schema with type and description
class MutualFund(BaseModel):
    scheme_name: str = Field(description="Name of the mutual fund scheme")
    fund_house: str = Field(description="Name of the fund house managing the scheme")
    scheme_type: str = Field(description="Type of the scheme, e.g. Open Ended or Close Ended Schemes")
    scheme_category: str = Field(description="Category of the scheme, e.g. Equity, Debt, Income")

class MutualFundOutput(BaseModel):
    reasoning: str = Field(description="Natural language explanation of the recommended portfolio and why it fits the user's needs")
    schemes: List[MutualFund] = Field(description="List of recommended mutual fund schemes")

parser = PydanticOutputParser(pydantic_object=MutualFundOutput)
```

Define Prompt

```
In [15]: ## TODO: Use ChatPromptTemplate.from_template with fields: preferences, context, format_instructions.
## Make sure no two selected mutual fund schemes fall under the same fund house AND scheme type.

template = ChatPromptTemplate.from_template(
    """You are a smart mutual fund investment advisor.

A user has described their investment needs below. Using ONLY the mutual fund schemes listed in the context, recommend a sensible investment portfolio that matches their preferences.
Make sure no two selected mutual fund schemes fall under the same fund house AND scheme type.

User preferences:
{preferences}

Available mutual fund schemes (from catalog search):
{context}

{format_instructions}
"""
)
```

Creating RAG Pipeline

Test RAG Pipeline Manually

```
In [16]: preference = "I want a low risk investment portfolio."
context_docs = retriever.invoke(preference) ## TODO: Retrieve relevant funds
relevant_text = "\n".join(doc.page_content for doc in context_docs) ## TODO: Joining the funds
llm = get_llm("GPT-OSS-120B (Groq)", 0.7) ## TODO: Get LLM
chain = template | llm | parser ## TODO: Create a Runnable Sequence with 3
```

Runnables - template, LLM and Output Parser

```
input_dict = {
    "preferences": preference,
    "context": relevant_text,
    "format_instructions": parser.get_format_instructions()
} ## TODO: Create dictionary with preferences, context and format_instructions
chain.invoke(input_dict) ## TODO: Invoke chain and the output of this command
should be a dictionary
```

```
Out[16]: MutualFundOutput(reasoning='The user wants a low-risk investment portfolio. Debt-oriented schemes such as Low Duration Funds and Short-Term Credit Risk Funds are generally the safest choices because they invest in high-quality, short-term instruments and have lower interest-rate and credit-risk exposure. To meet the additional constraint that no two selected schemes share both the same fund house and the same scheme type, each recommendation comes from a different fund house (all schemes are Open Ended, so avoiding duplicate fund houses automatically satisfies the rule). The three selected funds provide a diversified low-risk mix across different fund houses while staying within the debt category.', schemes=[MutualFund(scheme_name='Sundaram Short Term Credit Risk Fund - Dividend', fund_house='Sundaram Mutual Fund', scheme_type='Open Ended Schemes', scheme_category='Debt Scheme - Credit Risk Fund'), MutualFund(scheme_name='Franklin India Low Duration Fund - Quarterly - IDCW', fund_house='Franklin Templeton Mutual Fund', scheme_type='Open Ended Schemes', scheme_category='Debt Scheme - Low Duration Fund'), MutualFund(scheme_name='LIC MF Low Duration Fund-Regular Plan-Growth', fund_house='LIC Mutual Fund', scheme_type='Open Ended Schemes', scheme_category='Debt Scheme - Low Duration Fund')])
```

Define RAG Pipeline Function

```
In [17]: # RAG pipeline
def generate_portfolio(model_choice, user_input):
    context_docs = retriever.invoke(user_input["preferences"]) ## TODO:
    Retrieving relevant funds
    relevant_funds = "\n".join(doc.page_content for doc in context_docs) ##
    TODO: Joining the funds
    llm = get_llm(model_choice, user_input["temperature"]) ## TODO: Get the LLM

    #Invoking
    chain = template | llm ## TODO: Create a Runnable Sequence with 2 Runnables
    - template and LLM
    output = chain.invoke({
        "preferences": user_input["preferences"],
        "context": relevant_funds,
        "format_instructions": parser.get_format_instructions()
    }) ## TODO: Invoke the Runnable Sequence with the preferences, context and
    format instructions

    #Exception Handling
    try:
        result = parser.parse(output.content)
    except OutputParserException:
        return "Could not parse output.", None
    return result
```

User Interface

```
In [18]: def gradio_interface(preferences, model_choice, temperature):
    user_input = {
        "preferences": preferences,
        "model_choice": model_choice,
        "temperature": temperature,
    }
    result = generate_portfolio(model_choice, user_input)
    if not result or isinstance(result, str):
        return result, []
    if isinstance(result, tuple):
        explanation, _ = result
        return explanation, []
    # Build a plain text list of scheme recommendations
    list_output = "\n".join(
        f" ♦ {scheme.scheme_name} | Fund House: {scheme.fund_house}"
        for scheme in result.schemes
    )

    return result.reasoning, list_output
```

```

In [19]: # Gradio Blocks Layout
with gr.Blocks(title="Smart Mutual Fund Advisor") as demo:
    gr.Markdown("## Smart Mutual Fund Advisor")
    gr.Markdown("Enter your investment preferences and let AI suggest a portfolio!")

    with gr.Row():
        preferences_input = gr.Textbox(
            label="Describe your investment needs",
            placeholder="e.g., high return portfolio, only from Aditya Birla fund house",
            lines=2
        )

    with gr.Accordion("LLM Settings (Advanced)", open=False):
        model_dropdown = gr.Dropdown(
            label="Model",
            choices=model_choices,
            value=model_choices[0]
        )
        temperature_slider = gr.Slider(
            minimum=0.0, maximum=2, value=0.7, step=0.1,
            label="Temperature"
        )

    run_button = gr.Button("Generate Portfolio")

    with gr.Row():
        reasoning_output = gr.Textbox(label="Remarks", lines=2)

    with gr.Row():
        fund_list_output = gr.Textbox(label="Recommended Mutual Funds",
            lines=10)

    run_button.click(
        fn=gradio_interface,
        inputs=[preferences_input, model_dropdown, temperature_slider],
        outputs=[reasoning_output, fund_list_output]
    )

```

```

In [ ]: demo.launch(inline=True)

```

Part 2: Implement the RAG pipeline in LangGraph

Creating Graph

Define State Class

```

In [21]: ## TODO: Define State class with preferences, context and answer keys
class State(TypedDict):
    preferences: str

```

```
context: List[Document]
answer: MutualFundOutput
```

Define Nodes

```
In [22]: ## TODO: Define retrieve node
def retrieve(state: State):
    relevant_funds = retriever.invoke(state["preferences"]) ## TODO: Retrieving relevant mutual funds
    return {"context": relevant_funds} ## TODO: Return the relevant parts of the state: context only
```

```
In [23]: ## TODO: Define generate node
def generate(state: State):
    relevant_funds_str = "\n".join(doc.page_content for doc in state["context"]) ## TODO: Joining the mutual funds

    chain = template | llm_graph ## TODO: Create a Runnable Sequence with 2 Runnables - template and Llm_graph (different variable that's defined later)
    output = chain.invoke({
        "preferences": state["preferences"],
        "context": relevant_funds_str,
        "format_instructions": parser.get_format_instructions()
    }) ## TODO: Invoke the Runnable Sequence with the preferences, context and format instructions

    #Exception Handling for Output Parser
    try:
        result = parser.parse(output.content)
    except OutputParserException:
        return "Could not parse output.", None

    return {"answer": result} ## TODO: Return the relevant parts of the state: answer only
```

Build and Compile Graph

```
In [24]: ## TODO: Build the graph with 2 nodes and 3 edges
graph_builder = StateGraph(State) ## TODO: To add

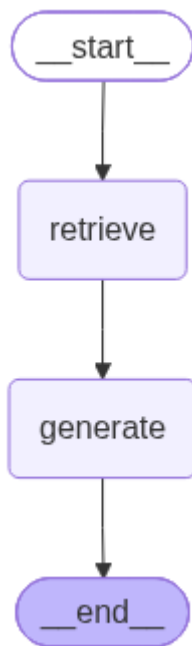
graph_builder.add_node("retrieve", retrieve) ## TODO: To add
graph_builder.add_node("generate", generate) ## TODO: To add

graph_builder.add_edge(START, "retrieve") ## TODO: To add
graph_builder.add_edge("retrieve", "generate") ## TODO: To add
graph_builder.add_edge("generate", END) ## TODO: To add
```

```
Out[24]: <langgraph.graph.state.StateGraph at 0x1c613caf770>
```

```
In [25]: #Compile graph
graph = graph_builder.compile()
```

```
In [26]: #Visualize graph
display(Image(graph.get_graph().draw_mermaid_png()))
```



Test Manually

```
In [27]: preferences = "I want a low risk investment portfolio."
llm_graph = get_llm("GPT-OSS-120B (Groq)", 0.7) ## TODO: Get LLM
result = graph.invoke({"preferences": preferences}) ## TODO: Invoke the graph
print(f'Context: {result["context"]}\n\n')
print(f'Answer: {result["answer"]}')

```

Context: [Document(id='453f974f-1ce0-49c9-a3be-77addbe8ee66', metadata={'scheme_category': 'Debt Scheme - Credit Risk Fund', 'scheme_type': 'Open Ended Schemes'}, page_content='Scheme Name: Sundaram Short Term Credit Risk Fund -Dividend | Fund House: Sundaram Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Debt Scheme - Credit Risk Fund | Net Asset Value: 11.3002 | Date: 29-12-2020'), Document(id='fecca05a-25c0-4f20-8a39-c8b955fd95fc', metadata={'scheme_type': 'Open Ended Schemes', 'scheme_category': 'Debt Scheme - Low Duration Fund'}, page_content='Scheme Name: Franklin India Low Duration Fund - Quarterly - IDCW | Fund House: Franklin Templeton Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Debt Scheme - Low Duration Fund | Net Asset Value: 14.7667 | Date: 07-08-2022'), Document(id='a7460221-d448-432e-b92f-68486846b0ac', metadata={'scheme_type': 'Open Ended Schemes', 'scheme_category': 'Equity Scheme - Small Cap Fund'}, page_content='Scheme Name: quant Small Cap Fund - IDCW Option - Regular Plan | Fund House: quant Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Equity Scheme - Small Cap Fund | Net Asset Value: 141.0509 | Date: 27-10-2023'), Document(id='c91f3b5f-51ad-4024-bd2f-18166a773c2e', metadata={'scheme_type': 'Open Ended Schemes', 'scheme_category': 'Debt Scheme - Low Duration Fund'}, page_content='Scheme Name: Franklin India Low Duration Fund - Monthly - IDCW | Fund House: Franklin Templeton Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Debt Scheme - Low Duration Fund | Net Asset Value: 15.0351 | Date: 07-08-2022'), Document(id='9072aebd-a02e-46fb-97f0-ce750856e2de', metadata={'scheme_type': 'Open Ended Schemes', 'scheme_category': 'Debt Scheme - Low Duration Fund'}, page_content='Scheme Name: LIC MF Low Duration Fund-Regular Plan-Growth | Fund House: LIC Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Debt Scheme - Low Duration Fund | Net Asset Value: 35.1561 | Date: 27-10-2023'), Document(id='01a3f488-ea36-458b-a7d8-7a930e8188c7', metadata={'scheme_type': 'Open Ended Schemes', 'scheme_category': 'Equity Scheme - Small Cap Fund'}, page_content='Scheme Name: quant Small Cap Fund - Growth - Regular Plan | Fund House: quant Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Equity Scheme - Small Cap Fund | Net Asset Value: 177.3894 | Date: 27-10-2023'), Document(id='69b5dcdd-76a2-4f68-90ea-ff355ff24e45', metadata={'scheme_category': 'Debt Scheme - Credit Risk Fund', 'scheme_type': 'Open Ended Schemes'}, page_content='Scheme Name: Sundaram Short Term Credit Risk Fund-Appreciation | Fund House: Sundaram Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Debt Scheme - Credit Risk Fund | Net Asset Value: 26.352 | Date: 29-12-2020'), Document(id='53316e62-06d1-4322-a274-07036f91a17d', metadata={'scheme_type': 'Open Ended Schemes', 'scheme_category': 'Debt Scheme - Low Duration Fund'}, page_content='Scheme Name: LIC MF Low Duration Fund-Regular Plan-Monthly IDCW | Fund House: LIC Mutual Fund | Scheme Type: Open Ended Schemes | Scheme Category: Debt Scheme - Low Duration Fund | Net Asset Value: 12.1226 | Date: 27-10-2023')]

Answer: reasoning='The user wants a low-risk investment portfolio. All three recommended schemes are debt-oriented, which generally exhibit lower volatility than equity funds. Each scheme comes from a different fund house, satisfying the requirement that no two selected funds share both the same fund house and scheme type (they all are Open Ended Schemes, so we avoid picking more than one from any single house). The chosen funds – a short-term credit risk fund from Sundaram, a low-duration fund from Franklin Templeton, and a low-duration fund from LIC – provide a blend of short-term credit exposure and stable, short-duration debt, suitable for a conservative investor.' schemes=[MutualFund(scheme_name='Sundaram Short Term Credit Risk Fund - Appreciation', fund_house='Sundaram Mutual Fund', scheme_type='Open Ended Schemes', scheme_category='Debt Scheme - Credit Risk Fund'), MutualFund(scheme_name='Franklin India Low Duration Fund - Monthly - IDCW', fund_house='Franklin Templeton Mutual Fund', scheme_type='Open Ended Schemes', scheme_category='Debt Scheme - Low Duration Fund'), MutualFund(scheme_name='LIC MF Low Duration Fund-Regular Plan-Growth', fund_house='LIC Mutual Fund', scheme_type='Open Ended Schemes', scheme_category='Debt Scheme - Low Duration Fund')]

User Interface

```
In [28]: def gradio_interface(preferences, model_choice, temperature):
    user_input = {
        "preferences": preferences,
        "model_choice": model_choice,
        "temperature": temperature,
    }
    llm_graph = get_llm(user_input["model_choice"], user_input["temperature"])
    result = graph.invoke({"preferences": preferences})["answer"]

    if not result or isinstance(result, str):
        return result, []
    if isinstance(result, tuple):
        explanation, _ = result
        return explanation, []
    # Build a plain text List of scheme recommendations
    list_output = "\n".join(
        f" ♦ {scheme.scheme_name} | Fund House: {scheme.fund_house}"
        for scheme in result.schemes
    )

    return result.reasoning, list_output
```

```
In [29]: # Gradio Blocks Layout
with gr.Blocks(title="Smart Mutual Fund Advisor") as demo:
    gr.Markdown("## Smart Mutual Fund Advisor")
    gr.Markdown("Enter your investment preferences and let AI suggest a portfolio!")

    with gr.Row():
        preferences_input = gr.Textbox(
            label="Describe your investment needs",
            placeholder="e.g., high return portfolio, only from Aditya Birla fund house",
            lines=2
        )

    with gr.Accordion("LLM Settings (Advanced)", open=False):
        model_dropdown = gr.Dropdown(
            label="Model",
            choices=model_choices,
            value=model_choices[0]
        )
        temperature_slider = gr.Slider(
            minimum=0.0, maximum=2, value=0.7, step=0.1,
            label="Temperature"
        )

    run_button = gr.Button("Generate Portfolio")

    with gr.Row():
        reasoning_output = gr.Textbox(label="Remarks", lines=2)

    with gr.Row():
        fund_list_output = gr.Textbox(label="Recommended Mutual Funds",
            lines=10)
```

```
run_button.click(  
    fn=gradio_interface,  
    inputs=[preferences_input, model_dropdown, temperature_slider],  
    outputs=[reasoning_output, fund_list_output]  
)
```

```
In [ ]: demo.launch(inline=True)
```